

DRAFT

From Primitive Provenance to Mathematical Structure

A Response-Coupled Derivation on a Growing b -ary Tree

Editor: Jan Seeck

Current draft: 6 August 2026

What CNNA means: ComplemeNt Net Architecture

The acronym **CNNA** expands to **ComplemeNt Net Architecture**. The internal capital “N” is deliberate: the project is not merely about a growing tree or a collection of later algebras. It studies one historically grown provenance *net* through a recurring architecture of relative complements.

At a chosen growth stage, cut, scale, or projection, the same net is resolved schematically into

realized or retained structure $\oplus$ relative complement $\oplus$ interface or handoff.

This display is architectural notation, not a claim that every later instance is a direct-sum decomposition. The mathematical realization changes with the node: an unrealized continuation slot, a Schur interior, a cut-relative environment, or information discarded by a later observation channel can occupy the complementary role.

The architecture begins at the origin. C001 fixes the empty baseline; C002 creates the first realized node; C003 and C018 distinguish the born prefix from admissible but not-yet-realized continuations; C004A selects the first open slot; and C013/C014 convert that slot into the first nontrivial weighted directed net. Thus CNNA starts with growth itself, before Schur complements, quantum language, or operator algebras appear.

The same concept then persists through the derivation:

open provenance slot $\rightarrow$ birth and directed relation,
 retained boundary / complementary interior $\rightarrow$ effective Schur/DtN response,
 immutable record / mutable live context $\rightarrow$ backreaction and memory,
 nested retained regions / nested complements $\rightarrow$ refinement and cocycle laws,
 provenance-complete state / reduced observation $\rightarrow$ later POVM, OQS, and AQFT specializations.

Complement is a relative role, not an intrinsic node label

CNNA does not assign every node one permanent opposite. What counts as retained, complementary, or interface depends on the growth stage, cut, scale, and observation map. A coordinate can be retained in one description and eliminated in another. The name therefore denotes an architectural invariant of the construction, not an additional set-theoretic axiom.

Formal status

This section fixes terminology and the project-wide interpretation. Its mathematical force remains exactly the force of the registered node contracts. In particular, it does not promote

the universal open-provenance hypothesis, the finite Legacy POVM, or later OQS/AQFT targets to proved results.

Conceptual scope: generalized open provenance systems

The project is organized around the following research hypothesis. An empirically open system may admit a *provenance-complete* extension in which its dynamically relevant history is represented as part of the real state, the observed subsystem is selected by a cut, and its effective response is obtained by eliminating or compressing the complementary degrees of freedom. In schematic form,

$$\mathfrak{P}_n = (\Gamma_n, \text{Rec}_n, \text{Live}_n, \mathcal{K}_n, \mathcal{C}_n, \mathcal{O}_n),$$

where Γ_n is an event-provenance DAG, Rec_n is immutable birth data, Live_n is the current response-bearing state, $\mathcal{C}_n$ is a cut, and $\mathcal{O}_n$ is an observation or coarse-graining channel. This is a *programmatic hypothesis*, not a theorem that every empirical open system possesses such a completion.

Formal status of the generalization

The current DAG proves only its registered finite CNNA specialization. The weak generalized hypothesis permits intrinsically stochastic provenance-complete dynamics. The stronger deterministic hypothesis claims that all apparent stochasticity is produced by partial observation, unresolved provenance, a cut-relative complement, coarse graining, or uncontrolled intervention. The stronger statement is not established by the present DAG.

For a retained region $B_n \subseteq V_n$, the complement $I_{B_n} = V_n \setminus B_n$ is interpreted as the environment relative to that cut. In the present finite directed-linear specialization, complement elimination is the exact unique-solve Schur/DtN operation

$$\Lambda_{B_n} = K_{BB} - K_{BI}X, \quad K_{II}X = K_{IB}.$$

This response reduction must not be identified with a quantum partial trace, a conditional expectation, or a stochastic marginalization. Those are mathematically different reductions that may instantiate the same abstract open-provenance role only after their own hypotheses are supplied.

The response-coupled CNNA update has the derived order

provenance state $\rightarrow$ canonical cut $\rightarrow$ effective response
 $\rightarrow$ steering $\rightarrow$ birth instruction $\rightarrow$ record/live update.

Later sections test the coherence of nested cuts, projective carriers, refinement maps, and cocycles. A finite response-derived POVM exists in the retained Legacy construction, but its migration into the derived-only DAG remains an explicit future proof chain:

positive compressed response state $\rightarrow$ frame effects $\rightarrow$ POVM completeness
 $\rightarrow$ Born evaluation $\rightarrow$ projective coarse graining.

No current node is relabelled as having proved a quantum instrument or an open-system limit.

Initial system-environment correlations show why a reduced state alone need not determine a universal completely positive reduced map. [Pec94, principal result]. Operational multi-time memory is naturally represented by a process object rather than by isolated one-step channels. [Pol+18, Abstract and process-tensor construction]. Quantum instruments separate outcome probabilities from outcome-conditioned state updates, [DL70, operational instrument framework]. while quantum networks and combs provide a later specialization of causally composable multi-step processes. [CDP09, quantum-network and comb framework]. These sources motivate the specialization map; none is imported as an axiom of the finite CNNA proofs.

Traceability and reading convention

Three coordinates, one scientific object

Every scientific node carries three distinct coordinates:

1. the canonical node number NNN, such as 001, which is the stable global presentation label in derivation order;
2. the semantic node identifier, such as I001 or C001, which identifies scientific ownership; and
3. the current section path, such as 1.1.1, which records only the present editorial location.

The visible node label is therefore

$$\text{NNN} \cdot \text{ID},$$

for example 001 · I001. The section path is deliberately excluded from this identity. A later insertion or reorganization of sections may change the path without renaming the DAG node, its semantic owner, or its node directory.

Canonical graph and generated projections

The files `derivation/registry/dag/NODES.tsv` and `EDGES.tsv` define the single semantic DAG. The editor-supplied yEd Live GraphML is the canonical visual layout of that same graph. The main-paper sequence, supplementary sequence, directory indices, and code-anchor tables are generated projections and are not parallel graphs.

For every node, `derivation/registry/nodes/<ID>.json` records the canonical number, semantic ID, current section path, documentation tier, document artifacts, dependencies, and code anchors.

Directory and code correspondence

The final node-directory component has the stable form

$$\text{NNN_ID_descriptive_slug}.$$

Its parent directories encode the current section hierarchy. Main-paper and supplementary artifacts mirror one another:

```
derivation/paper/main/sections/<parents>/NNN_ID__slug/SECTION.tex,
derivation/supplement/sections/<parents>/NNN_ID__slug/DOCUMENTATION.md.
```

Python modules have the form `sKK_id__slug.py`; Lean modules use `SKK_ID_Name.lean`. The prefix KK is a local module-order coordinate, not the canonical node number. The semantic ID is the authoritative join key.

The exact cross-layer mapping is stored in `CODE_ANCHORS.tsv`. A code citation consists of source path, symbol, declaration kind, line interval, and exact source SHA-256. Line numbers are reading aids; symbol plus hash is the stable identity.

Documentation tiers and completion state

A node's tier D0, D1, or D2 specifies the required depth: atomic definition, structural construction, or full proof dossier.

A node is marked COMPLETE_V2 only when its main-paper text, supplementary dossier, countercheck, code anchors, artifact hashes, and validation gates agree. Nodes are read in canonical order.

Reproducible lookup

To move from a DAG node to prose and code, read $\text{NNN} \cdot \text{ID}$, locate the ID in `NODES.tsv` or `NODE_TRACEABILITY_INDEX.tsv`, open the registered document artifact, and filter `CODE_ANCHORS.tsv` by the same ID. To move from code back to the DAG, confirm the source path and declaration symbol in the anchor table and follow its ID to the canonical node record.

This separation of identity, editorial placement, and implementation order is mandatory for all documentation tiers D0–D2.

Contents

What CNNA means: ComplemeNt Net Architecture	1
Conceptual scope: generalized open provenance systems	2
Traceability and reading convention	3
1 Primitive provenance and response-coupled growth	5
1.1 Primitive inputs, provenance grammar and event order	5
001 · I001 — Branching parameter b	5
002 · I002 — Finite approximant depth L	5
003 · C001 — Empty carrier $\emptyset$	6
004 · C002 — Root genesis r	7
005 · C003 — Finite b -ary provenance grammar	8
006 · C018 — Canonical BFS/lexicographic provenance birth schedule	10
007 · P002 — Canonical schedule strict-total-order closure	12
1.2 Bootstrap of the first provenance relation	13
008 · C004A — First provenance slot s_1	13
009 · A001 — Genesis seed $\star$	15
010 · N001 — Initial conductance normalization $C_\star = 1$	15
011 · C013 — First non-root provenance birth v_1	16
012 · T001 — Seed-neutrality theorem for the first birth	17
013 · M005 — Conductance-unit normalization independence	18
014 · C014 — Bootstrap state X_1	19
1.3 Canonical cut, complement elimination and birth steering	19
015 · C005 — Response-capable state schema $X_n, n \geq 1$	19
016 · C004 — Next open provenance slot $s_{n+1}, n \geq 1$	20
017 · C006 — Birth-local Schur/DtN primitive	21
018 · M001 — Canonical birth-local measurement cut $\mathcal{C}_n(s_{n+1})$	22
019 · P003 — Birth-cut canonicity and partition well-formedness	23
020 · M002 — Birth-cut interior-domain theorem	24
021 · C007 — Inter-birth directed response $R_n(s_{n+1})$	25
022 · O001 — IST response-independent directed-bias obstruction	26
023 · C015 — Active linear steering convention $\phi(x) = x$	27
024 · M003 — Canonical response-steering functional $\Sigma_b[R_n, s]$	27
025 · P001 — Reusable directed Schur/DtN/Kron channel closure	28
026 · M004 — Response-coupled birth law $\mathfrak{B}_b$	29

1. Primitive provenance and response-coupled growth

1.1. Primitive inputs, provenance grammar and event order

001 · I001 — Branching parameter b

Current section path: 1.1.1; documentation tier: D0.

Scientific role.. I001 is the first free structural input. It fixes only the number of available child slots at every provenance address; it is not inferred from a response, geometry, probability law, or later dynamical state.

Formal contract.. The admissible input domain is

$$\mathcal{B} := \{b \in \mathbb{N} \mid 2 \leq b\}. \quad (1)$$

A model instance must therefore supply one explicit $b \in \mathcal{B}$. No default value is part of the scientific contract. The exclusion of $b = 1$ is a domain declaration for the present CNNA family; this node does not claim that the unary case is logically inconsistent.

Executable and formal encoding.. Python stores the value in a frozen, field-explicit `BranchingParameter` and rejects any object that is not a built-in integer or is below the lower bound. Lean stores both the natural number and the proof $2 \leq b$ in the structure itself. Consequently, Python enforces the contract by checked construction, whereas Lean makes an invalid value uninhabitable without an accompanying proof. These are different mechanisms with the same admissible mathematical values.

Boundary countercheck.. The lower boundary $b = 2$ is admitted. Values $b < 2$ are rejected. Python also rejects `True` and `False`, even though `bool` is an implementation-level subclass of `int`; otherwise a logical flag could silently enter as a branching multiplicity. This is a type-boundary guard, not an additional mathematical axiom.

Result and handoff.. The node yields one validated branching multiplicity and nothing else. The hard derivation edge E001 hands b to C003, where the slot alphabet $\{0, \dots, b - 1\}$ is constructed. The later proof-support edge E147 supplies the same bound to P004; it adds no new content to the local contract.

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s01_i001_branching_parameter_b.py`, `BranchingParameter`, lines 20–46. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S01_I001_BranchingParameterB.lean`, `BranchingParameter`, lines 17–23; `lowerBound`, lines 24–27. The exact source hashes and test anchors are listed in the Supplement.

002 · I002 — Finite approximant depth L

Current section path: 1.1.2; documentation tier: D0.

Scientific role.. I002 is the second free structural input. It fixes the terminal provenance depth of the finite approximant. It is neither a spatial coordinate nor an event time, and it does not encode an infinite-limit sentinel.

Formal contract.. The admissible domain is

$$\mathcal{L} := \mathbb{N}_0, \quad L \in \mathcal{L}. \quad (2)$$

Thus every nonnegative integer is admissible, including $L = 0$. The value is supplied explicitly and no default is derived. At this node alone, $L = 0$ means only “zero permitted provenance extension depth”; the later root-only interpretation additionally uses the root construction C002 and the address grammar C003.

Executable and formal encoding.. Python uses a frozen `FiniteApproximantDepth` carrier and rejects negative or non-built-in-integer inputs. Lean uses a structure with one `Nat` field. No proof field is needed because `Nat` already represents exactly the required nonnegative values. The theorem `zeroAdmissible` records explicitly that zero is an ordinary member of the domain rather than a special missing-value marker.

Boundary countercheck.. $L = 0$ and arbitrary positive integers are admitted; negative integers, floating-point values, strings, `None`, and `Boolean` values are rejected at the Python boundary. No test or theorem in this node establishes a continuum or infinite-depth limit. Such a limit, if later constructed, is a separate downstream problem.

Result and handoff.. The node yields one validated finite cutoff. Edge E002 supplies it to C003; edge E081 later fixes the completion depth of C019; and proof-support edge E148 makes the same finite bound available to P004. None of these handoffs changes the meaning of L .

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s02_i002__finite_approximant_depth_l.py`, `FiniteApproximantDepth`, lines 20–46. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S02_I002_FiniteApproximantDepthL.lean`, `FiniteApproximantDepth`, lines 18–23; `zeroAdmissible`, lines 29–34. Exact hashes and tests are given in the Supplement.

003 · C001 — Empty carrier $\emptyset$

Current section path: 1.1.3; documentation tier: D0.

Scientific role.. C001 supplies the primitive pre-root state. Unlike I001 and I002, it is not a selectable numerical parameter: it is the unique carrier with no provenance content.

CNNA origin: the null baseline

ComplemeNt Net Architecture begins by making the pre-growth state explicit. C001 contains neither a realized node nor a hidden relation. This prevents the later retained/complement distinction from being smuggled into the initial condition: the first realized side of the architecture is created only by C002.

Formal contract.. Let V denote the set of present provenance nodes and $R \subseteq V \times V$ the present directed relations. The C001 carrier is

$$X_\emptyset = (V_\emptyset, R_\emptyset), \quad V_\emptyset = \emptyset, \quad R_\emptyset = \emptyset. \quad (3)$$

The contract asserts both existence of a pre-root carrier value and absence of nodes and relations. It introduces no root, address, event order, geometry, conductance, or response datum.

Executable and formal encoding.. Python represents $X_\emptyset$ by a frozen zero-field dataclass and the canonical value `EMPTY_CARRIER`; every node or relation query returns false. Lean uses an inhabited one-constructor inductive type, defines node and relation membership as `False`, and proves `eqCanonical`: every inhabitant equals the canonical value. The one-inhabitant program type represents uniqueness of the empty carrier; it is not an additional provenance node.

Empty mathematical content versus one canonical representation

In **Set**, the empty set is initial: for every set A there is exactly one map $\emptyset \rightarrow A$. [Lan98, Chs. I and III, categories and universal constructions]. The formal encoding here uses a singleton representation because the *state value* must exist and is unique, while its node and relation collections remain empty. A constructorless datatype would instead encode that no pre-root state exists at all, which is not the C001 contract.

Boundary countercheck.. The Python tests verify zero stored fields, empty node and relation tuples, vacuous membership predicates, and rejection of a hidden constructor payload. The Lean theorems `noNode`, `noRelation`, and `eqCanonical` separately establish emptiness and uniqueness. Thus the encoding cannot silently smuggle a root or initialization datum into C001.

Result and handoff.. The result is exactly one payload-free pre-root carrier. The sole outgoing hard edge E003 hands it to C002. Root existence and all root properties are therefore conclusions of the next construction, not assumptions embedded in the empty state.

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s03_c001__empty_carrier_empty.py`, `EmptyCarrier`, lines 23–52. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S03_C001_EmptyCarrierEmpty.lean`, `EmptyCarrier`, lines 21–26; `noNode`, lines 40–44; `noRelation`, lines 45–49; `eqCanonical`, lines 50–56.

004 · C002 — Root genesis r

Current section path: 1.1.4; documentation tier: D1.

Position and scientific role.. C002 is the first nonempty carrier construction. It consumes the unique empty carrier of C001 and creates the unique root token. It is deliberately prior to address grammar, event order, relation birth, geometry and response data; consequently none of those later structures may be used in the definition of genesis.

CNNA begins with the first realization

C002 is the first growth step of ComplemeNt Net Architecture: the empty carrier is replaced by the first realized provenance node. The root has no intrinsic permanent opposite. Its complementary continuation becomes meaningful only after C003 and C018 expose the admissible but not-yet-realized provenance slots.

Mathematical contract.. Let $X_\emptyset$ be the C001 carrier. Root genesis is the map

$$\Gamma_r : X_\emptyset \mapsto X_r \tag{4}$$

with

$$V(X_r) = \{r\}, \quad R(X_r) = \emptyset. \tag{5}$$

The root has no provenance parent and, at this derivation node, no geometric position. Equation (5) is stronger than merely asserting that a root exists: it fixes the entire local node and relation content immediately after genesis.

Explicit construction and invariants.. The construction introduces two singleton-valued objects: the zero-payload root token r and the canonical rooted carrier X_r . Node membership is therefore exactly the predicate $v = r$, whereas relation membership, parenthood of r , and geometric-position membership are false predicates. The construction preserves the following invariants:

$$|V(X_r)| = 1, \quad |R(X_r)| = 0, \quad \nexists p \text{ parent}(r) = p. \quad (6)$$

No address, numerical node identifier, sibling rank, birth index, conductance, load, response, or coordinate is introduced.

Canonicity.. Both the root type and the post-genesis carrier type have one constructor and no payload. Lean proves that every root value equals the canonical root and every rooted-carrier value equals the canonical carrier. Since C001 is also unique, `rootGenesis` is source-independent on its mathematical domain: there is no hidden root choice and no hidden initialization datum.

Boundary and counterchecks.. The executable checks separate genesis from several nearby but invalid interpretations. The result contains no self-relation (r, r) ; the root has no parent; both runtime carriers have zero stored fields; and a value outside the exact C001 carrier type is rejected rather than coerced. Reflection tests also exclude downstream metadata such as addresses, event indices, conductances and coordinates. These checks are not substitutes for the Lean uniqueness theorems; they test the Python boundary against accidental state smuggling.

Cross-layer realization.. Python uses frozen, slotted zero-field dataclasses `Root` and `RootedCarrier`; `root_genesis` checks its source type and returns the canonical carrier. Lean uses singleton inductive types, definitional false predicates for forbidden relations, parents and positions, and explicit uniqueness theorems. The mechanisms differ, but their observable contract is identical: one canonical node and no relation.

Result and handoff.. The hard edge E003 realizes the unique transition $C001 \rightarrow C002$. Edge E004 then hands the already born root carrier to C003, where the root is anchored to the empty address word. Thus root existence is proved before root addressing; the latter is not retroactively part of genesis.

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s04_c002_root_genesis_r.py`, `Root`, lines 21–26; `RootedCarrier`, lines 29–53; `root_genesis`, lines 56–65. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S04_C002_RootGenesisR.lean`, `Root`, lines 21–34; `RootedCarrier.ContainsNode`, lines 48–50; exclusion predicates, lines 52–65; uniqueness and genesis theorems, lines 67–113. Exact source hashes and test anchors are listed in the Supplement.

005 · C003 — Finite b -ary provenance grammar

Current section path: 1.1.5; documentation tier: D1.

Position and scientific role.. C003 is the first join of the three origin branches: I001 supplies $b \geq 2$, I002 supplies the finite cutoff $L \geq 0$, and C002 supplies the already-born canonical root. The node constructs the address grammar only. It does not construct an event schedule, node IDs, geometry, conductance, response, or dynamics.

CNNA grammar: realized prefix and open continuation

C003 gives the first structural form of the complement architecture. A born address prefix and its admissible right-extensions belong to one b -ary provenance grammar, while only a finite prefix is realized at a given growth stage. The complement here is the ordered family of admissible but not-yet-realized continuations, not a second externally supplied system.

Mathematical contract.. For a fixed branching parameter b , define the local slot alphabet

$$S_b := \{q \in \mathbb{N} \mid q < b\} = \{0, \dots, b-1\}. \quad (7)$$

The intrinsic address space is the free finite-word set S_b^* . The finite approximant admits

$$\mathcal{A}_{b,L} := \{a \in S_b^* \mid |a| \leq L\} = \bigcup_{d=0}^L S_b^d. \quad (8)$$

The C002 root is anchored to the empty word ε . For $u \in S_b^*$ and $q \in S_b$, intrinsic child formation is right extension

$$\text{snoc}(u, q) := u \parallel (q), \quad |u \parallel (q)| = |u| + 1. \quad (9)$$

Admission into the finite approximant is a separate condition: $|u| + 1 \leq L$.

The finite-word encoding and the induced rooted ordered tree are standard: concatenation creates a child and deletion of the final symbol recovers its parent. These textbook ingredients are not claimed as CNNA novelties; the CNNA-specific content is their exact provenance ownership, their separation from event order, and their role in the downstream derivation [M L97, Ch. 1 and free-monoid background]. [Die25, Ch. 1, The Basics].

Structural invariants.. The empty word has depth zero. Every non-root word admits a unique factorization

$$a = u \parallel (q), \quad q \in S_b, \quad (10)$$

which defines

$$\text{parent}(a) = u, \quad \text{rank}(a) = q. \quad (11)$$

Consequently parent, local rank and depth are derived from the address word; they are not independent fields. Equality $u \parallel (q) = v \parallel (s)$ forces both $u = v$ and $q = s$.

Cutoff separation and boundary cases.. The alphabet S_b and the operation snoc do not depend on L . The cutoff only determines which already well-formed words are admitted. In particular, $L = 0$ yields the root-only approximant $\mathcal{A}_{b,0} = \{\varepsilon\}$ while the intrinsic alphabet still contains all b slots. The root has neither a parent nor a final rank. Ranks outside S_b , malformed address carriers and extensions beyond L are rejected.

Structural comparison: finite von-Neumann ordinals and the unary chain

Extend the word grammar only for comparison to $S_1 = \{0\}$. Then there is exactly one address 0^d at each depth $d \leq L$, and the immediate-successor graph on $0, \dots, L$ is isomorphic to the rooted word chain $\varepsilon, 0, 00, \dots, 0^L$. Von Neumann's finite ordinals instead encode n as the transitive set of all predecessors. [Neu23, Introduction and displayed finite ordinals, pp. 199–200]. The comparison is therefore an exact graph isomorphism of successor skeletons, not an identification of words with ordinals. The active CNNA domain remains $b \geq 2$, where depth alone no longer identifies a node.

Cross-layer realization and proof logic.. Python represents words by validated tuples of built-in integers. Lean uses `List (Fin b.value)`, so slot admissibility is enforced by the element type itself. The central Lean proof chain is

$$\text{unsnoc?}_\text{snoc} \implies \{\text{parent?}_\text{snoc}, \text{finalSlot?}_\text{snoc}\} \implies \{\text{snoc_parent_unique}, \text{snoc_slot_unique}\} \quad (12)$$

while `depth_snoc` establishes the successor-depth law and `BoundedProvenanceAddress.child` transports it through the cutoff proof. Python tests exercise the same equations and rejection boundaries.

Countercheck and ownership boundary.. The implementation deliberately checks that the grammar object contains only the C002 rooted carrier, b , and L . Event IDs, birth times, linearization indices, node IDs, positions, conductances and responses are absent. This prevents a later schedule or physical field from being silently encoded in what should be a purely grammatical node.

Result and handoff.. C003 yields the finite rooted ordered address grammar $\mathcal{A}_{b,L}$ with canonical parent/rank decomposition. Edge E005 supplies it to C004A for the first provenance slot; E063 supplies it to C018 for the separate canonical birth schedule. The planned edge E402 records that a provenance-derived metric remains a later obligation; no metric follows merely from the word grammar.

Open-provenance role: Provenance-complete carrier

C003 supplies the finite event-address grammar of the current CNNA specialization. In the generalized open-provenance interpretation it is the carrier on which immutable event provenance can be recorded; it does not by itself assert that every empirical system has such a carrier.

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s05_c003_finite_b_ary_provenance_grammar.py`, word primitives, lines 32-96; bounded grammar, lines 99-173. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S05_C003_FiniteBaryProvenanceGrammar.lean`, slot and address types, lines 25-30; word construction and decomposition, lines 33-139; bounded addresses, lines 143-188; grammar join and root anchor, lines 192-240. The Supplement lists every definition, theorem, test block and exact source hash.

006 · C018 — Canonical BFS/lexicographic provenance birth schedule

Current section path: 1.1.6; documentation tier: D2.

Position and scientific role.. C018 consumes the finite b -ary address grammar of C003 and fixes the active deterministic order in which admissible non-root provenance slots are considered. It introduces no new free parameter. The root has already been created by C002; consequently it is not an element of the non-root birth schedule. The node owns an ordering convention only. It does not yet assign an event number, a birth time, a response value, a conductance, a geometry, or a dynamical update.

Formal order.. Let $S_b = \{0, \dots, b-1\}$ and let S_b^* be the C003 address words. Local slot ranks are ordered by the ordinary strict order on their values. At equal word length, this induces the lexicographic relation $<_{\text{lex}}$. C018 then defines

$$a \prec_{\text{BFSlex}} c \iff |a| < |c| \vee (|a| = |c| \wedge a <_{\text{lex}} c). \quad (13)$$

This is the shortlex order on provenance words: depth is compared first and lexicographic order breaks ties. It is selected as the active CNNA construction convention; C003 alone does not force this convention against all other possible linearizations. [M L97, Ch. 1, finite words and lexicographic order].

For one parent u and ranks $r < s$, the order satisfies

$$u \parallel (r) \prec_{\text{BFSlex}} u \parallel (s), \quad (14)$$

and every parent precedes every child,

$$u \prec_{\text{BFSlex}} u \parallel (r). \quad (15)$$

Shortlex order and the fixed-depth base- b coordinate

At depth d , an address $a = (a_0, \dots, a_{d-1})$ has the ordinary base- b coordinate $\sum_{j=0}^{d-1} a_j b^{d-1-j}$. Prefixing this coordinate by the number of words at smaller depths yields the closed index

$$1 + \sum_{k=1}^{d-1} b^k + \sum_{j=0}^{d-1} a_j b^{d-1-j}$$

for the non-root shortlex schedule. Leading zero slots remain part of the word, so this is a coordinate for the chosen order rather than an identification of provenance words with integers. [M L97, Ch. 1].

Strict-total-order theorem.. The Lean development proves constructively that the address relation in Eq. (13) is irreflexive, transitive, asymmetric, and trichotomous:

$$a \prec_{\text{BFSlex}} c \vee a = c \vee c \prec_{\text{BFSlex}} a. \quad (16)$$

Hence distinct provenance addresses are comparable in exactly one direction. The proof first establishes irreflexivity, transitivity and trichotomy for the lexicographic word relation and then lifts these facts through the depth-first disjunction defining BirthBefore. No classical case split is needed.

Bounded open slots.. For a C003 grammar with cutoff L , the formal slot object consists of a bounded parent u , a local rank r , and a proof that $|u| + 1 \leq L$. Its selected child is $u \parallel (r)$. The slot relation is defined by comparing these child addresses with $\prec_{\text{BFSlex}}$. It is therefore irreflexive, transitive and asymmetric; slots selecting distinct children are totally comparable. The word “open” at this node means *admissible potential birth slot*. Openness relative to an evolving born set and uniqueness of the least currently open slot require additional state and are owned by C004. P002 certifies only the static order exported by C018; its source awaits the local kernel build.

Executable realization and cross-layer agreement.. Python stores one slot as $(u, r, u \parallel (r))$ and orders it by

$$K(u, r) = (|u|, u, r). \quad (17)$$

A breadth-first parent queue emits ranks $0, \dots, b-1$ in increasing order and appends each emitted child to the same queue. Because every child has length $|u| + 1$ and lexicographic comparison of $u \parallel (r)$ first compares u and then r , the key in Eq. (17) is extensionally the same ordering criterion as the Lean child-address relation. The executable tests reproduce exact schedules for $(b, L) = (2, 3)$ and $(3, 2)$, strictness, transitivity, determinism, cutoff zero, and the finite carrier cardinality.

Necessity checks and ownership boundary.. Removing the depth comparison would produce depth-first lexicographic order, not breadth-first order. Removing the final rank from the Python key would leave siblings of one parent unordered. Treating the root as a birth slot would duplicate the C002 genesis. Conversely, the ordering relation alone does not prove that a state-dependent selector possesses a unique least open slot, nor that the finite schedule enumerates every bounded address exactly once. Static order certification is isolated in P002, state-dependent least-open selection is owned by C004, and finite exhaustivity by P004; C018 is not promoted beyond its current source theorems.

Result and downstream handoff.. C018 yields a fixed, strict total order on provenance addresses and a derived strict order on admissible slots. It selects the first slot for C004A, orders recurrent candidates for C004, supplies the ordering input later used by C010, C019, C042 and the control CTRL005, and supports the proof gates P002, P003 and P004. These outgoing edges expose dependencies; they do not close the downstream nodes.

Open-provenance role: Event provenance versus recurrent live dynamics

C018 orders birth events in an acyclic provenance history. Later live-state feedback may be recurrent or cyclic without turning the event-provenance relation itself into a cycle.

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s01_primitive_inputs_grammar_and_event_order/s06_c018__canonical_bfs_lexicographic_provenance_birth_schedule.py`, `open_slot_key`, lines 46-53; `canonical_birth_slots`, lines 61-88; `CanonicalBirthSchedule`, lines 97-121. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S06_C018_CanonicalBfsLexicographicProvenanceBirthSchedule.lean`, order definitions, lines 23-37; address-order proof chain, lines 39-236; bounded-slot realization, lines 241-325. The complete declaration and hash register is given in the Supplement.

007 · P002 — Canonical schedule strict-total-order closure. *Current section path: 1.1.6.1; documentation tier: D2.*

Position and proof responsibility.. P002 is the static proof-certification child of C018. It exports one stable contract for the breadth-first/lexicographic provenance order. The contract contains no born-state, unsaturation, least-open, or executable-selector hypothesis. Those state-dependent statements are owned by C004 after C005 has introduced the response-capable state.

Formal contract.. For provenance addresses a, c , C018 defines the strict order

$$a \prec c \iff |a| < |c| \vee (|a| = |c| \wedge a \prec_{\text{lex}} c).$$

P002 packages irreflexivity, transitivity, asymmetry, trichotomy, and comparability of distinct addresses. For C018 open-slot records it packages irreflexivity, transitivity, and asymmetry of the induced order. Because a slot record contains proof-bearing cutoff data, totality is stated extensionally:

$$s \prec_{\text{slot}} t \vee \text{child}(s) = \text{child}(t) \vee t \prec_{\text{slot}} s.$$

Thus P002 claims a strict total order on selected child addresses, not proof-irrelevant equality of arbitrary slot records.

Minimal selected-child uniqueness.. For an explicitly supplied predicate Q on C018 slot records, define

$$\text{Minimal}_Q(s) \iff Q(s) \wedge \forall t [Q(t) \Rightarrow \neg(t \prec_{\text{slot}} s)].$$

P002 proves that two Q -minimal witnesses select the same child address. The proof uses the C018 comparison theorem for distinct selected children: either one witness precedes the other, contradicting the corresponding minimality clause, or their selected children are equal.

Formal realization.. The dedicated proof module imports exactly the mathlib-free C018 owner module and consumes its public order theorems explicitly. It exports one closure structure, one construction theorem, the minimality predicate and uniqueness theorem, and the stable proposition-valued contract. The complete declaration list is given in the Supplement. No Python module is required because P002 introduces no executable selector.

Necessity and scope checks.. The extensional equality alternative is necessary: two proof-bearing records may select one child without being definitionally equal. Total comparison is necessary for minimal-witness uniqueness. Conversely, a static total order does not define the least address not yet born; that theorem requires the C005 born prefix and remains at C004. P002 therefore creates no backward dependency on recurrent-state data.

Trust boundary and current result.. The source contains no project-local axiom, sorry, native_decide, Classical, simp, or simpa. The dedicated Lean 4.31.0 build and #print axioms audit remain the only open gate. Until that local build succeeds, P002 is active and unfrozen; no kernel-verification claim is made.

Downstream handoff.. The successful contract certifies the C018 owner and supplies the static order interface required by P004 for finite schedule exhaustivity. P003 does not consume P002: birth-cut canonicity already depends directly on C018 and C004.

Code line register. derivation/code/lean/proofs/src/CNNAProofs/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S01_PrimitiveInputsGrammarAndEventOrder/S06_C018_CanonicalBfsLexicographicProvenanceBirthSchedule/Proofs/S01_P002_CanonicalScheduleStrictTotalOrderClosure.lean, lines 19–65 (closure structure), 69–99 (closure construction), 104–109 (minimality predicate), 115–131 (extensional uniqueness), 134–135 (public contract), and 138–140 (contract witness).

1.2. Bootstrap of the first provenance relation

008 · C004A — First provenance slot s_1

Current section path: 1.2.1; documentation tier: D1.

Position and scientific role.. C004A is the first node in the bootstrap group. It consumes the finite b -ary provenance grammar from C003 and the canonical breadth-first/lexicographic order from C018. Its sole task is to select the first *structural provenance slot*. It does not perform a birth, add a vertex to the carrier, assign a time, or introduce conductance, response or geometry.

CNNA handoff: selecting the first open complement slot

C004A selects the least admissible continuation of the realized root without yet turning it into a node. It is therefore the first explicit handoff from an open complement possibility to a birth input. C013, not C004A, owns the actual realization and relation.

Explicit construction.. Let ε denote the C003 root address and let the active branching domain satisfy $b \geq 2$. The least local C003 rank is the zero-based slot $q_0 = 0 \in S_b$. The first provenance slot is therefore

$$s_1 := (\varepsilon, q_0, \varepsilon \parallel q_0), \quad \text{parent}(s_1) = \varepsilon, \quad \text{rank}(s_1) = 0, \quad \text{addr}(s_1) = (0). \quad (18)$$

The subscript in s_1 is a one-based ordinal name: “the first slot.” It does not change the stored zero-based sibling rank.

The formal object carries exactly the two direct predecessors, a C003 grammar G and a C018 schedule Σ , together with the coherence condition $\Sigma.\text{grammar} = G$. Parent, rank and address are derived functions, not independently supplied fields.

Structural invariants and minimality.. The address has depth one and C003 recovers both of its syntactic components:

$$|\text{addr}(s_1)| = 1, \quad \text{parent?}(\text{addr}(s_1)) = \varepsilon, \quad \text{finalSlot?}(\text{addr}(s_1)) = 0. \quad (19)$$

Rank zero is the least local slot. C018 therefore places s_1 before every distinct sibling of the root. More strongly, for every non-root provenance address c ,

$$c = (0) \quad \vee \quad (0) \prec_{\text{BFSlex}} c. \quad (20)$$

For depth-one addresses this is the increasing-rank branch of C018; for depth at least two it follows from breadth-first depth priority. Thus (0) is the unique minimum non-root address under the already-proved strict order. This is a theorem about the structural address space, not yet about the first accepted birth event.

Cutoff boundary.. Finite admission is deliberately separated from structural existence. Define

$$\text{WithinCutoff}(s_1) \iff |\text{addr}(s_1)| \leq L. \quad (21)$$

Since the depth is exactly one,

$$L \geq 1 \implies \text{WithinCutoff}(s_1), \quad L = 0 \implies \neg \text{WithinCutoff}(s_1). \quad (22)$$

At $L = 0$ the intrinsic word (0) and the label s_1 remain well-defined, but the finite approximant is root-only and C018 enumerates no admissible birth slot. This prevents the cutoff from being mistaken for part of the local branching grammar.

Cross-layer realization.. Python stores the two predecessors in an immutable dataclass and computes parent, rank, address and cutoff admission as properties. It rejects predecessor objects of the wrong type and rejects a C003 grammar paired with a C018 schedule built from another grammar. Lean stores the same predecessors in `FirstProvenanceSlot` and internalizes their coherence as the proof field `schedule_grammar`. The Lean lemmas `address_parent` and `address_finalSlot` prove the syntactic recovery equations; `address_eq_or_before_nonroot` and `address_before_distinct_nonroot` prove global minimality; the two cutoff theorems prove the exact $L = 0$ versus $L \geq 1$ boundary.

Counterchecks and ownership boundary.. The Python suite checks the one-based/zero-based distinction, all active sample values $b \in \{2, 3, 5\}$ and $L \in \{0, 1, 3\}$, first-position agreement with the C018 enumeration for positive cutoff, rejection of incoherent predecessors, and absence of geometry, node IDs, event indices, times, conductance and response fields. These tests exclude both an off-by-one slot and accidental promotion of a provenance label into a physical or dynamical state.

Result and downstream handoff.. C004A closes the canonical structural selection

$$s_1 = (\varepsilon, 0, (0)), \quad (23)$$

with (0) minimal among non-root addresses and admitted exactly when $L \geq 1$. Edges E005 and E064 are the verified incoming handoffs from C003 and C018. Verified edge E007 supplies this slot to C013, where the first non-root vertex and first directed relation are constructed. No such birth is owned by the present node.

Primary code anchors. Python: `derivation/code/python/src/cnna/derivation/s01_primitive_response_coupled_finite_provenance/s02_bootstrap_of_the_first_relation/s01_c004a__first_provenance_slot_s1.py`, class and derived properties, lines 36–82; constructors, lines 85–99; tests, lines 19–100. Lean: `derivation/code/lean/core/src/CNNA/Derivation/S01_PrimitiveResponseCoupledFiniteProvenance/S02_BootstrapOfTheFirstRelation/S01_C004A_FirstProvenanceSlotS1.lean`, predecessor structure and construction, lines 23–45; address invariants and minimality, lines 46–168; cutoff boundary, lines 169–185. The Supplement lists all 23 declaration-level anchors and their exact source hashes.

009 · A001 — Genesis seed *

Current section path: 1.2.2; documentation tier: D0.

Position and role.. A001 is an auxiliary root of the canonical DAG. It is introduced at this point because the exceptional constructor C013 is the first node that requires an explicit bootstrap argument. A001 is not derived from C004A; the two objects meet only as independent inputs of C013.

Definition.. The genesis-seed carrier is the singleton

$$\mathcal{S}_\star := \{\star\}. \quad (24)$$

The token has no numerical, address, conductance, response, timing, geometric, or dynamical component. It is therefore neither a third free model parameter nor an initial physical state.

Formal encoding and countercheck.. Python represents the carrier by an immutable zero-field dataclass. Distinct runtime instances compare equal because they have the same empty payload. Lean uses a one-constructor inductive type and proves

$$\forall \eta : \mathcal{S}_\star, \quad \eta = \star. \quad (25)$$

The node-local test audits the empty field list and explicitly checks that the canonical token exposes none of the downstream result fields. A payload-bearing seed would violate A001 even if it happened to be initialized with one value.

Result and handoff.. A001 closes only the existence of one explicit, information-free bootstrap token. Verified edge E006 hands that token to C013. The claim that the first-birth result is independent of the supplied token belongs to T001, not to A001.

Primary code anchors. Python source and test: `class GenesisSeed`, lines 14–15, and `TestGenesisSeed`, lines 13–24. Lean: `GenesisSeed`, `canonical`, and `eqCanonical`, lines 12–27. The Supplement binds all five declaration-level anchors to their exact source hashes.

010 · N001 — Initial conductance normalization $C_\star = 1$

Current section path: 1.2.3; documentation tier: D0.

Position and role.. N001 is a fixed-normalization root of the DAG, introduced immediately before the first relation is constructed. It is independent of A001 and C004A and supplies only the numerical representative used by C013.

Definition.. The bootstrap conductance unit is fixed by

$$C_{\star} := 1, \quad (C_{r \rightarrow v_1}, C_{v_1 \rightarrow r}) := (1, 1). \quad (26)$$

The equality of the two orientations is an initialization convention for the first relation. N001 does not create either endpoint or the relation itself. The local natural-number carrier of the numeral 1 is only the representation used at this node; it does not prescribe the scalar carrier of all later conductances and responses.

Formal encoding and countercheck.. Both Python and Lean encode the normalization as a singleton-valued, zero-payload object. Python rejects an attempted constructor argument such as “2”; Lean’s one-constructor type has no field through which an alternative value could enter. The functions `value` and `directedValues` then return 1 and (1, 1) definitionally. This establishes fixedness but not unit-independence: the latter requires comparison with alternative positive representatives and is proved only in M005.

Result and handoff.. Verified edge E008 supplies the pair (1, 1) to C013; verified edge E010 supplies the canonical representative to M005. N001 contributes no free model input and no birth-state payload.

Primary code anchors. Python: `InitialConductanceNormalization`, lines 23–34, and its test, lines 14–29. Lean: singleton type, canonical value, scalar value, directed pair and the unit-pair theorem, lines 15–37. The Supplement lists all seven anchors with exact hashes.

011 · C013 — First non-root provenance birth v_1

Current section path: 1.2.4; documentation tier: D1.

Position and scientific role.. C013 joins three independent bootstrap inputs: the structural first slot C004A, the singleton token A001, and the fixed unit normalization N001. It is exceptional because before this step there is no nontrivial weighted relation and hence no Schur/DtN response from which a response-coupled birth could be computed.

First nontrivial ComplemeNt Net

C013 realizes the first selected open slot and joins the newborn to the existing root by the first weighted directed relation. This is the first nontrivial *net* in CNNA. Complementarity is relational and stage-dependent: root and newborn are not declared permanent opposites, but jointly form the first realized response context from which later relative cuts can be taken.

Mathematical contract and construction.. Let

$$s_1 = (\varepsilon, 0, (0)) \quad (27)$$

be the C004A slot and assume its address lies in the finite cutoff. C013 constructs

$$v_1 = (0), \quad \text{parent}(v_1) = r = \varepsilon, \quad (28)$$

with the two stored directed relations and conductances

$$\mathcal{R}_1 = ((r, v_1), (v_1, r)), \quad \mathcal{C}_1 = (1, 1). \quad (29)$$

The output stores the slot, the normalization and, in Lean, the proof of cutoff admission. The A001 token is consumed as a constructor argument but is not a field of the resulting state.

Invariants and boundary.. The newborn address is definitionally the C004A address, its syntactic parent is the root, and both conductances are inherited from N001. No event index, birth time, geometry, response, steering value or later live-state update is created. Since $| (0) | = 1$, the construction exists for $L \geq 1$ and is rejected at $L = 0$. The structural word (0) continues to exist at $L = 0$; only its promotion to a born carrier element fails.

Cross-layer realization and counterchecks.. Python checks exact predecessor types and executes the cutoff guard before constructing an immutable two-field birth object. Lean moves the cutoff guard into the input proposition and stores its proof in the structure. Both layers derive the endpoints, relation orientations and unit pair from the same predecessors. The Python tests check the complete first relation, the $L = 0$ rejection and the absence of a retained seed field. These checks exclude both an illicit response-driven bootstrap and hidden information transfer from A001.

Canonicity boundary, result and handoff.. For the fixed C004A slot and N001 normalization, the endpoint and conductance data are canonical. C013 does not itself state the equality of outputs under two explicit seed arguments; T001 owns that theorem. Verified edge E009 sends the construction to T001 and verified edge E011 sends the completed birth to C014.

Primary code anchors. Python construction: lines 23-63; focused tests: lines 17-33. Lean structure, constructor, derived endpoints, relation pair, conductance pair and three invariant theorems: lines 22-77. The Supplement records all 13 anchors and source hashes.

012 · T001 — Seed-neutrality theorem for the first birth

Current section path: 1.2.5; documentation tier: D2.

Formal statement.. For a fixed admissible first slot s_1 , fixed N001 normalization N , and any two seed values $\eta, \eta' \in \mathcal{S}_*$, let $B(s_1, \eta, N)$ denote the C013 constructor. T001 proves

$$B(s_1, \eta, N) = B(s_1, \eta', N) \quad \wedge \quad \mathcal{C}(B(s_1, \eta, N)) = (1, 1). \quad (30)$$

Proof mechanism.. The result is stronger than a post-hoc comparison of the unique A001 value: the C013 constructor does not use its seed argument when assembling the output. Consequently the state equality is definitional (rfl). The second conjunct is inherited from C013's unit-conductance theorem. No quotient, choice principle or extensional reconstruction is needed.

Executable and formal evidence.. Python creates two distinct runtime instances of the zero-field seed class, passes them through the same constructor and checks equality of the resulting birth records and their conductances. Lean quantifies over arbitrary GenesisSeed values and proves the conjunction directly. Thus the Python test exercises object-level erasure, while Lean supplies the universal state-equality theorem.

Necessity checks and scope.. If C013 stored the seed, used it to select an address, or used it to modify a conductance, the definitional equality proof would fail. Merely knowing that A001 is a singleton would then require a separate rewrite by seed uniqueness; the current proof does not need such a rewrite. T001 concerns only the exceptional first birth. It does not establish conductance-unit independence, recurrent seed-neutrality, or any later response property.

Axiom profile, result and handoff.. The theorem is in the mathlib-free core and is proved by `rfl` plus the already-built C013 theorem; the current source contains no project axiom, sorry, admit or opaque proof declaration. Verified edge E012 certifies the seed-neutrality component of C014.

Primary code anchors. Python wrapper and test: lines 15–21 and 16–27. Lean seed-indexed constructor and theorem: lines 15–41. The Supplement gives the four exact anchors and their hashes.

013 · M005 — Conductance-unit normalization independence

Current section path: 1.2.6; documentation tier: D2.

Formal statement.. N001 chooses $C_\star = 1$ as the canonical bootstrap unit. M005 defines equality of normalized scalar responses without division by

$$(R, C) \sim (R', C') \iff RC' = R'C. \quad (31)$$

For every rational scale $\lambda > 0$,

$$(R, C) \sim (\lambda R, \lambda C), \quad (32)$$

and for every positive rational unit u and normalized value x ,

$$(x, 1) \sim (ux, u). \quad (33)$$

Proof strategy and interpretation.. Equation (32) is the associativity and commutativity identity $R(\lambda C) = (\lambda R)C$. Positivity is not needed for that algebraic identity; it identifies the rescaling as an admissible change of a positive conductance unit. [DS84, Electrical network preliminaries]. Python evaluates the corresponding quotient exactly with `Fraction` and rejects nonpositive units and scales. Lean uses `Rat` and the cross-product relation, avoiding division and all regularization.

Necessity checks.. Scaling only R or only C generally changes the normalized value. A zero scale satisfies the bare cross-product identity but cannot represent a unit, and a negative scale violates the positive-conductance convention even though it preserves an algebraic ratio. These distinctions prevent the theorem from being misread as an unrestricted gauge symmetry.

Positive scaling classes, not a gauge field

The positive pairs (R, C) and $(\lambda R, \lambda C)$ lie on one orbit of the multiplicative action of $\mathbb{Q}_{>0}$, and R/C is constant on that orbit. Choosing $C_\star = 1$ selects a convenient representative. [DB13, Sec. I, current-balance and reduced conductance equations]. No local gauge group, gauge connection, edge-dependent rescaling symmetry or dynamical gauge field is introduced at this node.

Result, limits and handoff.. M005 proves that the fixed N001 numeral is a unit representative rather than a third physical input. The theorem is scalar and rational and makes no claim of homogeneity for the full Schur/DtN map. Verified edge E013 certifies C014. Verified edge E068 supplies the fixed representative $C_\star = 1$ consumed by M003 after the C015 identity transform.

Primary code anchors. Python exact quotient and rescaling checks: lines 19–46; test: lines 14–25. Lean cross-product relation, N001 rational lift and two rescaling theorems: lines 19–73. The Supplement lists all nine anchors and exact hashes.

014 · C014 — Bootstrap state X_1

Current section path: 1.2.7; documentation tier: D1.

Position and scientific role.. C014 closes the exceptional bootstrap section. It packages the concrete C013 birth and imports the two independent certificates T001 and M005. The result is the base case handed to the recurrent state schema C005.

First response-capable CNNA state

C014 packages the first weighted two-node net. From this point onward the complement architecture can become dynamical: later nodes may retain part of the net, eliminate a complementary interior, read the resulting response, and feed it into the next birth. C014 itself performs none of those later operations; it supplies their minimal nontrivial carrier.

Explicit construction.. The mathematical contents are

$$X_1 = (V_1, \mathcal{R}_1, \mathcal{C}_1), \quad V_1 = \{r, v_1\}, \quad \mathcal{R}_1 = ((r, v_1), (v_1, r)), \quad \mathcal{C}_1 = (1, 1), \quad (34)$$

with $r = \varepsilon$ and $v_1 = (0)$. In the implementation this tuple is not stored redundantly: `BootstrapState` contains exactly one field, the verified C013 birth, and exposes the endpoints, relations and conductances by projection.

Invariants and certificates.. C014 inherits the root, newborn and unit-pair invariants from C013. Lean then lifts T001 through the wrapper, proving that changing the explicit seed leaves X_1 unchanged. It also rationally lifts the two actually stored conductances, identifies both with the N001 representative, and applies M005 separately to each orientation. The comparison scale is not stored in X_1 and no second bootstrap state is constructed.

Boundary cases and counterchecks.. C014 can be built only after C013, hence only when $L \geq 1$. At $L = 0$ the root-only approximant exists but no X_1 does. Python rejects a non-C013 payload and verifies that the dataclass has only the field `birth`. Neither layer stores the seed, a variable unit, a response value, a time, geometry or recurrent-update data. “Response-capable” means only that a nontrivial weighted relation now exists on which later response operators can act.

Cross-layer agreement, result and handoff.. Python realizes the minimal wrapper and executable projections. Lean realizes the same wrapper and additionally attaches theorem-level seed and unit certificates without adding runtime fields. Verified edge E014 provides the recurrent base case to C005. The later finite-iteration handoffs E077 and E153 remain planned and blocked; C014 by itself proves no recurrent existence theorem.

Primary code anchors. Python state, projections, constructor and test: lines 23–51 and 18–30. Lean wrapper, projections, seed-neutrality lift, rational conductance lift and unit-representative theorem: lines 23–99. The Supplement lists all 12 anchors with exact source hashes.

1.3. Canonical cut, complement elimination and birth steering**015 · C005 — Response-capable state schema X_n , $n \geq 1$**

Current section path: 1.3.1; documentation tier: D1.

After the exceptional bootstrap, a recurrent state X_n consists of the fixed finite provenance grammar and canonical birth schedule, the first $n \geq 1$ non-root addresses of that schedule, and the current positive directed conductances supported on the born carrier. The born addresses

therefore form an initial segment of the C018 order. Every born non-root vertex retains both directed positive conductances to its provenance parent; additional positive directed conductances between already-born vertices are permitted.

If the provenance labels are temporarily forgotten, X_n is a finite positively weighted digraph with a distinguished bidirectional spanning parent backbone. Standard digraph language therefore applies to paths and connectivity, while the schedule-prefix and provenance invariants remain CNNA-specific.

[BG09, Ch. 1, pp. 1–30].

The bootstrap state of C014 embeds as the base case X_1 . Thus the root and first newborn are present with the two unit conductances inherited from the bootstrap construction. The recurrent rational conductance fields are the explicit rational lifts of those two stored C014 values, rather than fresh unit literals; hence the C014-to-C005 scalar handoff is term-level.

This node defines only the domain schema for recurrent growth. It does not select the next open slot, choose the birth-local cut, compute a Schur/Dirichlet-to-Neumann response, apply a birth law, or construct X_{n+1} . In particular, “response-capable” here means that a finite positive weighted provenance carrier is available as input to the later response construction; the response itself is owned by subsequent DAG nodes.

Open-provenance role: Sufficient finite provenance state

C005 is the current finite candidate for a provenance-sufficient state: it carries the born prefix and response data needed by the next CNNA update. Whether an arbitrary empirical reduced state admits such a completion is outside the present theorem scope.

Verified closure and limits.. The formal state invariants require a nonempty C018 prefix, pairwise distinct born addresses, pairwise distinct directed conductance records, positive non-loop conductances with born endpoints, and both parent orientations for every born non-root vertex. The constructor `fromBootstrap` proves that C014 inhabits this schema with $n = 1$. C005 is a state domain only: it contains neither a selected successor nor a response witness or update law.

016 · C004 — Next open provenance slot s_{n+1} , $n \geq 1$

Current section path: 1.3.2; documentation tier: D1.

Let X_n be a recurrent response-capable state from C005 and let $S_{b,L} = (s_1, \dots, s_N)$ denote the finite canonical provenance-slot order fixed by C018. Since the born non-root carrier of X_n is exactly the initial C018 prefix of length n , the next open provenance slot is, for $1 \leq n < N$,

$$s_{n+1} := S_{b,L}[n]$$

with zero-based implementation indexing. Equivalently, s_{n+1} is the unique admissible open slot whose child has no admissible open C018 predecessor. Thus every admissible address strictly before its child is already born, while the child itself is not born.

The child address determines its provenance parent and final sibling rank uniquely by the finite grammar of C003. Hence the relational characterization by the least open child address and the explicit provenance slot (parent, rank, child) identify the same mathematical object.

If $n = N$, the finite approximant is saturated and no s_{n+1} exists. No sentinel, out-of-cutoff address, geometric birth location, event index, response quantity, conductance update, or birth law is introduced at this node.

Open-provenance role: Open slot as state-relative incompleteness

C004 makes openness relative to a current born prefix: the next slot is absent from the present state but admissible in the fixed provenance grammar. This is the finite CNNA specialization of openness relative to an incomplete state description.

Verified closure and limits.. Lean constructs the least open address by finite enumeration of the already fixed C003 carrier and proves existence under *Unsaturated*, uniqueness, parent bornness, and nonexistence under *Saturated*. Python computes the same object by schedule indexing and checks it against an independent extensional least-open predicate. No separate schedule is introduced.

017 · C006 — Birth-local Schur/DtN primitive

Current section path: 1.3.3; documentation tier: D1.

Let an explicitly supplied finite operator be written in an already fixed ordered boundary/interior basis as

$$K = \begin{pmatrix} K_{BB} & K_{BI} \\ K_{IB} & K_{II} \end{pmatrix},$$

where the boundary block is nonempty. C006 owns only the algebraic elimination primitive. It neither selects the cut nor constructs K from the current CNNA conductances.

An interior solve is an encoded matrix X satisfying the exact fraction-value equation

$$K_{II}X = K_{IB}.$$

The C006 admissibility domain is unique solvability of this equation. On that domain the Schur/DtN response is

$$\Lambda = K_{BB} - K_{BI}X.$$

This is the standard Schur-complement elimination pattern. In network language, the same boundary operator is called a Dirichlet-to-Neumann response, and elimination of interior network nodes is called Kron reduction. C006 contributes the exact partial-value formulation and the strict ownership boundary, not the underlying block identity.

[Zha05, Horn-Zhang, Basic Properties, pp. 17-46].

[DB13, Sec. I, Eqs. (1)-(2), pp. 150-151].

[CIM98, Abstract and Sec. 1, p. 115].

The definition is row-by-column and uses the supplied coordinate order without permutation or implicit transpose. No symmetrization, floating tolerance, condition-number threshold, pseudoinverse, or regularization belongs to C006. For empty interior the unique solve is the empty matrix and the formula reduces to $\Lambda = K_{BB}$.

One block operation, four disciplinary names

The matrix $K_{BB} - K_{BI}K_{II}^{-1}K_{IB}$ is called a Schur complement in matrix analysis and appears as the result of Gaussian elimination of the interior variables.

[DB13, Sec. I, pp. 150-151].

For a conductance network it is the boundary voltage-to-current or Dirichlet-to-Neumann response; eliminating interior network nodes is commonly called Kron reduction.

These are not four separate mechanisms. They are four semantic views of the same block elimination. CNNA's distinctive step is to make the boundary/interior split birth-local and provenance-derived before applying the standard operator.

Status. Exact Algebraic Common Core Different Semantics. The common algebraic core is exact; application-specific assumptions remain distinct and are tracked separately.

Lean carries the finite arithmetic on positive-denominator pairs (z, d) and identifies two pairs by cross multiplication,

$$(z, d) \sim (z', d') \iff zd' = z'd.$$

Reflexivity, symmetry, and transitivity of this relation are proved, as are the congruence of addition, multiplication, subtraction, finite folded sums, matrix multiplication, matrix subtraction, and response replacement. Hence C006 is internally well defined on exact fraction values rather than on one particular numerator-denominator representation. Core Rat is used only

as a canonical positive-denominator encoding of supplied entries and solve witnesses; C006 does not claim a theorem equating its operations with `Rat.add`, `Rat.mul`, or `Rat.sub`.

Python uses normalized `Fraction` values and exact Gauss-Jordan elimination. Agreement between the Python implementation and the Lean fraction-value algebra is therefore an explicit cross-language verification obligation, supported by exhaustive operand tests, matrix-product checks, and independent state-level rational calculations; it is not disguised as a Lean theorem about core rational arithmetic. The response relation itself is normalization invariant: admissibility yields existence and uniqueness modulo entrywise fraction-value equivalence.

Exact elimination before floating-point linear algebra

Exact linear elimination predates modern arbitrary-precision libraries. Bareiss's integer-preserving elimination is a classical example: algebraic identities are organized so intermediate divisions remain exact.

[Bar68, pp. 565–578].

C006 uses a different construction—positive-denominator fraction values modulo cross multiplication and an exact unique-solve relation—but shares the same methodological refusal to replace algebraic equality by a floating tolerance.

This matters scientifically because a condition-number threshold or pseudoinverse would not merely accelerate the calculation; it would change the domain and therefore the mathematical object being derived.

Status. Methodological Lineage Not Algorithm Identity. The comparison is methodological; C006 correctness rests entirely on its own exact-value proofs.

The canonical birth-local instantiation is deliberately deferred. M001 supplies the ordered cut, M002 states its exact domain, and C007 constructs the current directed operator from C005 and applies this C006 primitive to obtain the pre-birth response.

Open-provenance role: Exact complement elimination

C006 realizes one specific open-provenance reduction: exact directed Schur/DtN elimination by a unique interior solve. It is not a partial trace, marginalization, or quantum instrument.

Verified closure and limits.. The core module proves that exact admissibility yields response existence and value-level uniqueness, and that the zero-interior case is always admissible. Its internal positive-denominator fraction algebra identifies raw encodings by cross multiplication. The later P001 package proves the semantic bridge to ordinary rational matrix arithmetic; C006 itself does not import that later proof or make the partial operator total.

018 • M001 — Canonical birth-local measurement cut $\mathcal{C}_n(s_{n+1})$

Current section path: 1.3.4; documentation tier: D2.

Let X_n be a recurrent state admitted by C005 and let $s_{n+1} = (p, \rho, c)$ be the unique next open provenance slot identified by C004. Here p is the already-born provenance parent, ρ its local sibling rank, and $c = p \frown \rho$ the not-yet-born child address.

M001 defines birth locality using only the finite provenance relations already present at this stage. Write

$$A(p) = \{\epsilon, p|_1, \dots, p\}$$

for the root-to-parent prefix chain, and

$$S_{<}(s_{n+1}) = \{p \frown j : 0 \leq j < \rho\}$$

for the earlier same-parent sibling slots. These are the two provenance roles directly resolved by the next slot: its causal prefix chain and the already resolved local ranks preceding it. No weights are used in their definition.

Let

$$V_n^{\text{ord}} = (\epsilon, v_1, \dots, v_n)$$

denote the C005 carrier in its inherited C018 order. Define the ordered boundary and interior by restriction of this already-fixed order,

$$B_n(s_{n+1}) = V_n^{\text{ord}}|_{A(p) \cup S_{<}(s_{n+1})}, \quad I_n(s_{n+1}) = V_n^{\text{ord}}|_{V_n \setminus B_n(s_{n+1})},$$

and set

$$\mathcal{C}_n(s_{n+1}) = (B_n(s_{n+1}), I_n(s_{n+1})).$$

Thus M001 does not introduce a second ordering convention: each block is an ordered subsequence of the C005 carrier.

By the C005 initial-prefix property and the C004 least-open characterization, p and every earlier sibling $p \frown j$ with $j < \rho$ are already born. The prefix ancestry of p is already born by provenance-parent closure. Hence $B_n(s_{n+1}) \subseteq V_n$. The C004 child c is absent from V_n , so it occurs in neither block. Boundary and interior are disjoint and exhaust the entire born carrier. Consequently the displayed rule determines a unique finite ordered cut from (X_n, s_{n+1}) .

The word “canonical” is relative to this frozen provenance selector: M001 is the minimal cut retaining exactly the causal-prefix and earlier-sibling roles of the next slot. It does not claim that no other provenance-defined cut can be written down. In particular, M001 introduces no sampling cap, geometric radius, terminal frontier, response-dependent selection, or downstream spacetime notion.

The cut is still purely combinatorial. C006 acts only after such an ordered boundary/interior pair has been supplied, and M002 owns the separate question whether the corresponding interior block lies in the exact Schur/DtN domain.

Open-provenance role: Cut-relative system and environment

M001 selects retained boundary ports and an eliminated interior. The interior is the environment relative to this cut, not a second primitive substance; changing the cut changes the system/environment roles.

Proof-owner closure.. The owner module contains the complete P003 certificate: both lists are born-only, disjoint, exhaustive on the born carrier, order-preserving because they are filters of the canonical carrier, uniquely determined by the frozen selector, and exclude the unborn child. Thus the earlier extension marker is retired for M001 itself; later general-cut node P007 remains a separate future generalization.

019 · P003 — Birth-cut canonicity and partition well-formedness. *Current section path: 1.3.4.1; documentation tier: D2.*

Let X_n be a C005 state, let s_{n+1} be its C004 successor, and let $\mathcal{C}_n(s_{n+1}) = (B_n, I_n)$ be the M001 cut. P003 certifies that this construction is a genuine ordered partition of the already-born carrier:

$$B_n \cap I_n = \emptyset, \quad B_n \cup I_n = V_n, \quad c_{n+1} \notin B_n \cup I_n.$$

Every entry of both lists is born. The two lists preserve C005/C018 order because M001 defines them by filtering the canonical carrier rather than by a new sorting operation. The predicate *IsCanonicalCut* identifies exactly the two filtered lists, so any two canonical witnesses are equal.

The certificate is implemented as internal theorems of the M001 owner module, not as a second mathematical construction or a parallel proof DAG. The relevant theorem chain is

`boundary_node_born, interior_node_born, boundary_interior_disjoint, carrier_covered, chi`

C004 directly supplies the verified least-open successor, while C018 supplies the order needed by M001. P003 therefore closes the M001 owner gate without a P002 dependency.

020 · M002 — Birth-cut interior-domain theorem

Current section path: 1.3.5; documentation tier: D2.

Let X_n be a recurrent C005 state, let s_{n+1} be its C004 successor, and let

$$\mathcal{C}_n(s_{n+1}) = (B_n, I_n)$$

be the canonical ordered M001 cut. M001 fixes the coordinate sets and their order, but it does not assign numerical entries to the four C006 blocks. Consequently the cut alone cannot determine whether an interior matrix is invertible or, equivalently for the C006 square interior system, whether

$$K_{II}X = K_{IB}$$

has a unique exact fraction-value solution.

M002 therefore closes the second branch of its obligation: it states the exact domain rather than inserting an unproved global invertibility hypothesis.

This partiality is standard linear-algebraic hygiene: a Schur complement is not made total merely by notation. Replacing the unique-solve domain with a pseudoinverse or regularization would define a different response operator and therefore a different model.

[Zha05, Basic Properties, pp. 17–46]. For explicit C006 block data K whose dimensions agree with the M001 cut, define

$$\mathcal{D}_{\text{M002}} = \{(X_n, s_{n+1}, K) : \dim K = (|B_n|, |I_n|) \text{ and } K \in \text{Dom}(\text{C006})\}.$$

Here $K \in \text{Dom}(\text{C006})$ means exactly that the interior system has one and only one encoded solve under the C006 fraction-value equation. No floating tolerance, condition-number threshold, pseudoinverse, regularization, positivity surrogate, or matrix convention is substituted for this predicate.

The nonempty M001 boundary required by C006 is itself derived: the provenance root is always on the M001 causal predecessor chain, hence $|B_n| > 0$. If $I_n = \emptyset$, C006 already gives a unique empty solve, so every block assignment of the matching zero-interior shape belongs to the M002 domain.

For $|I_n| > 0$, M001 alone cannot imply membership in the domain. Indeed, for the same fixed M001 dimensions choose all non-interior blocks zero and $K_{IB} = 0$. With $K_{II} = I$ the unique solve is $X = 0$; with $K_{II} = 0$ every matrix X solves the homogeneous system, and because both $|B_n|$ and $|I_n|$ are positive there are at least two such solves. Thus one and the same combinatorial cut admits both an admissible and a non-admissible numerical block assignment. This obstruction is precisely why C007, not M001 or M002, must construct the actual directed operator from the C005 conductances.

Python checks the exact M001 block dimensions and delegates acceptance exactly to the C006 fraction-value admissibility predicate. Lean enforces the same dimensions in the type of BirthCutBlocks, proves the M001 boundary nonempty, defines InExactDomain to be exactly C006 admissibility, proves the zero-interior case, and transports domain membership to existence and uniqueness of the exact C006 response value modulo raw-fraction normalization. The two languages therefore agree on the same exact partial domain; the Python contrast witness is additional executable evidence for the cut-only obstruction, not a different definition.

Verified closure and limits.. The theorem fixes the exact handoff rather than claiming universal invertibility. Boundary nonemptiness follows from the root port; zero interior is unconditionally admissible; nonempty interiors require the stated C006 unique-solve predicate. The Python contrast witness demonstrates that identical M001 dimensions can support both admissible and inadmissible block entries.

021 · C007 — Inter-birth directed response $R_n(s_{n+1})$

Current section path: 1.3.6; documentation tier: D1.

Let X_n be a recurrent C005 state with $n \geq 1$, let s_{n+1} be its unique C004 next open provenance slot, and let

$$\mathcal{C}_n(s_{n+1}) = (B_n, I_n)$$

be the canonical ordered M001 cut. The C005 current carrier is partitioned exhaustively by $B_n \sqcup I_n$, while the unborn child of s_{n+1} belongs to neither block.

For each stored directed conductance $c_n(u, v) > 0$, C007 fixes the source/out-degree matrix convention

$$K_n(u, u) = \sum_v c_n(u, v), \quad K_n(u, v) = -c_n(u, v) \quad (u \neq v).$$

The row coordinate is therefore the conductance source and the column coordinate its target. This is not a tunable sign convention: it is the direct matrix realization of the ordered pair (u, v) already stored by C005. No transpose, in-degree alternative, symmetrization, geometric factor, external port, or response-independent bias is inserted.

The resulting matrix is a standard nonsymmetric directed Laplacian: off-diagonal entries are nonpositive and every row sums to zero. The scientific choice that matters here is not the existence of such a Laplacian, but the fixed source/out-degree orientation inherited from the stored conductance pairs.

[AC05, Abstract and Sec. 1].

Order the complete born carrier as B_n followed by I_n and write

$$K_n = \begin{pmatrix} K_{BB}^{(n)} & K_{BI}^{(n)} \\ K_{IB}^{(n)} & K_{II}^{(n)} \end{pmatrix}.$$

Because this order is inherited from M001, C007 introduces no new coordinate permutation. On the exact M002 domain, equivalently when

$$K_{II}^{(n)} X = K_{IB}^{(n)}$$

has a unique exact fraction-value solution, the inter-birth directed response is

$$R_n(s_{n+1}) = K_{BB}^{(n)} - K_{BI}^{(n)} X.$$

This is precisely the C006 Schur/DtN response of the C005 state-dependent block realization.

The directed Laplacian is minus a Markov generator

Define $Q_n := -K_n$. Then for $u \neq v$, $Q_n(u, v) = c_n(u, v) \geq 0$, while $Q_n(u, u) = -\sum_v c_n(u, v)$ and every row sums to zero.

[Nor97, Ch. 2, continuous-time Markov chains].

This is exactly the sign pattern of a continuous-time Markov-chain generator. The same directed weights can therefore be read either as conductances in K_n or transition rates in Q_n .

The comparison is deliberately one-way: CNNA does not thereby introduce probabilities, waiting-time distributions, or a stochastic clock. It exposes a reusable algebraic bridge for later analysis.

Status. Exact Sign Duality Without Stochastic Postulate. The sign duality is exact under the C007 row/source convention; any stochastic interpretation would require additional normalized state semantics.

The temporal statement is literal: $R_n(s_{n+1})$ is measured after the first n non-root births and before birth $n + 1$. The next child is not a coordinate of K_n or R_n . C007 does not yet convert this matrix into a scalar steering functional and does not execute a growth update; those operations belong to M003 and M004.

The construction remains partial exactly on the M002 domain. C007 neither replaces exact unique solvability by a condition-number threshold nor asserts a stronger invertibility theorem without proof. For the finite C005 fixtures covered by the executable audit, including asymmetric conductances and extra born-born directed edges, all 99 tested reachable pre-birth states lie in that exact domain.

Open-provenance role: Effective response after complement elimination

C007 is the retained-port response of the pre-birth live state. It is the finite directed-linear effective dynamics associated with the M001 cut, before scalar steering or event creation.

Exact-value output semantics.. The displayed matrix is an elementwise exact fraction value. A Lean raw fraction matrix is identified internally by cross multiplication. Python returns normalized Fraction representatives; their agreement with Lean is an explicit executable cross-language verification, not a theorem about core Rat arithmetic. Response uniqueness is therefore value uniqueness, not structural equality of a particular numerator-denominator encoding.

Verified closure and limits.. The implementation proves response existence and value-level uniqueness on exact M002 domain membership and excludes the unborn child from both block coordinate sets. C007 fixes only the pre-birth matrix response. Scalar steering, robustness transforms, directed birth bias, and the transition to X_{n+1} remain downstream.

022 · 0001 — IST response-independent directed-bias obstruction

Current section path: 1.3.7; documentation tier: D2.

The retained legacy growth path contains executable scalar mechanisms that can change birth data without being functions of the measured pre-birth response $R_n(s_{n+1})$. O001 classifies eight represented mechanism classes:

$$\chi = (\chi_{\text{rank}}, \chi_{\rightarrow}, \chi_{\leftarrow}, \chi_{\text{load}}, \chi_{\text{nonlin}}, \chi_{\text{scale}}, \chi_{\text{base}}, \chi_{\text{atten}}) \in \{0, 1\}^8.$$

The admission predicate is not heuristic:

$$\text{Admissible}(X, s, R, \Sigma, \chi) \iff \chi = (0, 0, 0, 0, 0, 0, 0, 0).$$

On this branch the obstruction record is erased and the output is exactly the four-tuple $(X_n, s_{n+1}, R_n, \Sigma_b)$. Each nonzero component is rejected individually; the complete legacy witness is rejected as a special case.

The executable audit is source-bound. Python inspects executable AST nodes in the retained legacy functions and ignores comments and docstrings. Lean formalizes the same eight-field presence record, proves preservation of the four admitted fields, and proves one rejection theorem for every channel. The prefix-free root package containing this module was built successfully under Lean 4.31.0 in the user-reported 26-job Core build.

O001 is therefore a closed implementation obstruction, not a universal no-go theorem for every conceivable growth law. It neither chooses the C015 transform nor proves M003 positivity. Its downstream statement is narrower: the active M004 constructor may depend on provenance support and on the response-derived scalar, but not on any of the eight represented independent mechanism classes.

Established ingredients, nonstandard derivation chain

Most individual operations used so far are established: finite rooted trees, finite words and lexicographic order, weighted digraphs, Schur complements, Dirichlet-to-Neumann response, and directed Kron reduction all have independent literatures. [Die25, Ch. 1].

[M L97, Ch. 1].

[Zha05, Basic Properties, pp. 17–46].

[SS23, Definition 3.2, Lemmas 3.3–3.4, Theorem 3.9].

The CNNA-specific claim lies in the composition: the carrier begins from the empty object; addresses own provenance; shortlex fixes the birth prefix; the next slot determines a birth-local boundary; the current directed state determines an exact response; and response-independent legacy coefficients are then excluded before steering is admitted.

Thus the work should not be read as claiming a new tree, a new Schur complement, or a new directed Laplacian. Its proposed novelty is the derived ownership chain and the insistence that no downstream datum be inserted before its provenance node has supplied it.

This distinction is reviewer-relevant: textbook citations justify the local steps, while the DAG, cross-layer locks, and obstruction gates are the evidence for the nonstandard global assembly.

Status. Literature Positioned Novelty Boundary. This positioning is an interpretive boundary, not a theorem. The global novelty claim remains contingent on completeness and coherence of the full derived chain.

023 • C015 — Active linear steering convention $\phi(x) = x$

Current section path: 1.3.8; documentation tier: D0.

C015 fixes one active-path convention,

$$\phi_A : S \rightarrow S, \quad \phi_A(x) = x,$$

for the exact scalar type supplied by M003. It is not a selectable mode family and introduces no slope, coefficient, coercion, normalization, clipping, logarithm, saturation, sign change, or fallback branch. Python returns the same object without conversion; Lean proves both pointwise identity and equality with the identity endomorphism by definitional reduction.

The boundary check is deliberately structural: the public API has one positional argument and exports no mode or scale object. Null, logarithmic, and saturated transforms remain separately named controls and are not hidden alternatives inside C015. The sole handoff is the unchanged scalar to M003.

024 • M003 — Canonical response-steering functional $\Sigma_b[R_n, s]$

Current section path: 1.3.9; documentation tier: D2.

Let $s = s_{n+1}$ be the canonical C004 slot and $p = p(s)$ its parent. In the M001 boundary order, M003 extracts the exact parent-port diagonal aggregate

$$R_p(\Lambda, s) = \sum_{i: \partial_i = p} \Lambda_{ii}.$$

The boundary is duplicate-free, so the address-extensional sum selects one port without a free coordinate. With $C_\star = 1$ and $\phi_A = \text{id}$,

$$\Sigma_b[\Lambda, s] = R_p(\Lambda, s).$$

The Core proves exact existence, value uniqueness, and invariance under C006 representative replacement. P001 supplies the reusable directed Schur-DtN/Kron theorem and canonical-cut positivity. The closed proof interface CanonicalM003Closure obtains the parent coordinate internally and proves, for every canonical realization, response-domain inhabitation, existence of a response-steering pair, and strict positivity of every such pair. No caller supplies a parent index or positivity hypothesis.

This is the kernel-verified M003 handoff consumed by M004. The four public closure declarations were compiled with Lean 4.31.0 and passed the registered axiom audit.

Open-provenance role: Response-to-event specialization

M003 converts the effective boundary response into the unique positive scalar used by the next event. This is the response-coupling step of the current deterministic specialization, not a universal law for all open systems.

025 · P001 — Reusable directed Schur/DtN/Kron channel closure. *Current section path: 1.3.9.1; documentation tier: D2.*

P001 is the unique proof owner behind the C006-C007-M003-M004 channel. The mathlib-free Core supplies exact finite directed states, cuts, responses, steering values, and the birth-law relation. The separate pinned CNNAProofs package proves the analytic and finite-combinatorial closure without introducing a Core-to-proof import.

Formal contract.. For an ordered boundary/interior block family, P001 assumes only non-positive off-diagonal entries, exact row conservation, positive interior-to-boundary reachability, and a positive path from the distinguished boundary port to a different boundary port. It proves exact Core-to-rational semantic agreement, existence and uniqueness of the interior solve, response existence and witness independence, the directed-Laplacian sign and conservation laws, nonnegative response diagonal, and strict positivity of the distinguished diagonal. No symmetry, inverse, pseudoinverse, regularization, extra grounding vertex, or independent connectivity axiom is added.

Proof architecture.. The proof sequence is explicit. S01 identifies the Core exact-fraction and matrix operations with rational values. S02 proves the directed maximum principle and triviality of the zero-boundary interior kernel. S03 uses the finite-dimensional injectivity-surjectivity bridge to obtain the unique interior solve. S04 proves response existence and representative independence. S05 proves nonpositive off-diagonal response entries, row conservation, and diagonal nonnegativity. S06 derives strict distinguished positivity by propagation along a positive path to a second boundary port. S07-S09 derive the canonical parent coordinate, exact C007 matrix laws, and provenance-backbone reachability directly from C005, M001, M002, and C007. S10 exposes the P001-to-M003/M004 supporting theorems without changing either Core constructor. The public closure modules above P001 consume these theorems. S11 independently instantiates the same generic closure on a two-boundary/one-interior bidirected-chain family.

For the independent family with strictly positive rational weights g_L, g_R ,

$$K_{BB} = \text{diag}(g_L, g_R), \quad K_{BI} = \begin{pmatrix} -g_L \\ -g_R \end{pmatrix}, \quad K_{IB} = (-g_L \quad -g_R), \quad K_{II} = (g_L + g_R).$$

Its sign, row-sum, and path hypotheses are proved directly. The closure theorem then invokes only directedSchurDtNClosure. The reuse therefore does not depend on ResponseCapableState, the next slot, provenance addresses, M001, or a C007 realization.

Kernel evidence and trust boundary.. The user-local Lean 4.31.0 run built 26 Core jobs and 8595 proof jobs across 13 proof modules. All 142 registered P001 declarations passed P001_CURRENT_PROOF_AXIOM_AUDIT, and the final FULL_PACKAGE_BOUNDARY_AUDIT passed with zero P001 source warnings. The exact transcript SHA-256 is 3329291658b2d7a5f46acc6c1bf48b8a60f6bade5d010aa96d7978be3943170a. The axiom-profile partition is 117 declarations with propext, Classical.choice, and Quot.sound; 23 with propext and Quot.sound; and two axiom-free declarations. No project-local axiom or sorry occurs. Elimination of the three transitive Lean/mathlib axioms is not claimed.

[The26a, Subsingleton.elim declaration, Lean 4.31.0 Init.Core].

[The26c, rational constructor and arithmetic lemmas].

[The26b, finite-fold recursion equations].

[Lea26a, finite-sum recursion equations].
 [Arl+26, rectangular row-by-column multiplication definition].
 [Lea26b, `Finset.sum_eq_zero_iff_of_nonneg`].
 [Lea26c, `Finset.exists_max_image`].
 [Lea26d, `Fintype.sum_sum_type`, `Fintype.sum_eq_single`, `Fintype.sum_eq_zero`].
 [Höl+26, `Matrix.mulVecLin`, `Matrix.mulVecLin_apply`].
 [Hug26, finite-dimensional injectivity-surjectivity equivalence].
 [BA26, module header, `Matrix.PosSemidef` and `Matrix.PosDef` definitions].
 [Ben+26, module header and main results].
 [SS23, Definition 3.2, Lemmas 3.3–3.4, Theorem 3.9].

Open-provenance role: Reusable elimination theorem

P001 certifies the directed Schur/DtN/Kron specialization independently of the canonical birth cut. Its generality is mathematical reuse across admissible finite cuts; it does not identify Schur reduction with other open-system reductions.

026 • M004 — Response-coupled birth law $\mathfrak{B}_b$

Current section path: 1.3.10; documentation tier: D1.

For the canonical pre-birth realization, M004 consumes the closed M003 interface and transports the exact steering scalar σ unchanged to the provenance-determined instruction:

$$\begin{aligned} c(p, x) &= c(x, p) = \sigma, \\ c(x, a) &= \sigma && (a \text{ a strict ancestor of } p), \\ c(q, x) &= c(x, q) = \sigma && (q \text{ an older sibling}), \\ \ell_{n+1} &= \sigma. \end{aligned}$$

The Core retains a zero-inclusive algebraic lift and a strictly positive active constructor. The proof layer no longer asks a caller for positivity or a parent coordinate: `canonicalM004Closure` uses `CanonicalM003Closure` directly to construct an active instruction, prove uniqueness for each exact response-steering pair, and prove representative independence. It exposes `IsCanonicalBirthInstructionHandoff` as the immutable output boundary for C008. C008 alone owns record/live state mutation.

No second birth-law constructor, regularization, clipping, bias, load, rank, or extra scalar is introduced. The seven public M004 closure and handoff declarations were compiled with Lean 4.31.0 and passed the registered axiom audit.

Open-provenance role: Birth instruction before record/live mutation

M004 turns the positive response scalar into a representative-independent provenance instruction. C008 remains responsible for applying that instruction to the immutable record and mutable live channels.

References

- [AC05] Rafiq Agaev and Pavel Chebotarev. “On the Spectra of Nonsymmetric Laplacian Matrices”. In: *Linear Algebra and its Applications* 399 (2005). published article; arXiv v1; arXiv v1; arXiv DOI 10.48550/arXiv.math/0508176, pp. 157–168. DOI: 10.1016/j.laa.2004.09.003. arXiv: math/0508176. URL: <https://doi.org/10.1016/j.laa.2004.09.003>.
- [Arl+26] Ellen Arlt, Blair Shi, Sean Leather, Mario Carneiro, Johan Commelin, and Lu-Ming Zhang. *Mathlib module: Data.Matrix.Mul*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.

- [BA26] Alexander Bentkamp and Mohanad Ahmed. *Mathlib module: Matrix.PosDef*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [Bar68] Erwin H. Bareiss. “Sylvester’s Identity and Multistep Integer-Preserving Gaussian Elimination”. In: *Mathematics of Computation* 22.103 (1968). published article, pp. 565–578. DOI: [10.1090/S0025-5718-1968-0226829-0](https://doi.org/10.1090/S0025-5718-1968-0226829-0). URL: <https://doi.org/10.1090/S0025-5718-1968-0226829-0>.
- [Ben+26] Alexander Bentkamp, Eric Wieser, Jeremy Avigad, and Johan Commelin. *Mathlib module: Matrix.SchurComplement*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [BG09] Jørgen Bang-Jensen and Gregory Z. Gutin. *Digraphs: Theory, Algorithms and Applications*. 2nd ed. Springer Monographs in Mathematics. second edition. Springer London, 2009. 795 pp. DOI: [10.1007/978-1-84800-998-1](https://doi.org/10.1007/978-1-84800-998-1). URL: <https://doi.org/10.1007/978-1-84800-998-1>.
- [CDP09] Giulio Chiribella, Giacomo Mauro D’Ariano, and Paolo Perinotti. “Theoretical framework for quantum networks”. In: *Physical Review A* 80.2 (2009). published article, p. 022339. DOI: [10.1103/PhysRevA.80.022339](https://doi.org/10.1103/PhysRevA.80.022339). URL: <https://doi.org/10.1103/PhysRevA.80.022339>.
- [CIM98] Edward B. Curtis, David Ingerman, and James A. Morrow. “Circular Planar Graphs and Resistor Networks”. In: *Linear Algebra and its Applications* 283.1–3 (1998). published article; author-hosted manuscript, pp. 115–150. DOI: [10.1016/S0024-3795\(98\)10087-3](https://doi.org/10.1016/S0024-3795(98)10087-3). URL: [https://doi.org/10.1016/S0024-3795\(98\)10087-3](https://doi.org/10.1016/S0024-3795(98)10087-3).
- [DB13] Florian Dörfler and Francesco Bullo. “Kron Reduction of Graphs with Applications to Electrical Networks”. In: *IEEE Transactions on Circuits and Systems I: Regular Papers* 60.1 (2013). published article; arXiv v1; arXiv DOI 10.48550/arXiv.1102.2950, pp. 150–163. DOI: [10.1109/TCSI.2012.2215780](https://doi.org/10.1109/TCSI.2012.2215780). arXiv: [1102.2950](https://arxiv.org/abs/1102.2950). URL: <https://doi.org/10.1109/TCSI.2012.2215780>.
- [Die25] Reinhard Diestel. *Graph Theory*. 6th ed. Graduate Texts in Mathematics 173. sixth edition. Springer Berlin, Heidelberg, 2025. 455 pp. ISBN: 978-3-662-70107-2. DOI: [10.1007/978-3-662-70107-2](https://doi.org/10.1007/978-3-662-70107-2). URL: <https://doi.org/10.1007/978-3-662-70107-2>.
- [DL70] E. B. Davies and J. T. Lewis. “An operational approach to quantum probability”. In: *Communications in Mathematical Physics* 17.3 (1970). published article, pp. 239–260. DOI: [10.1007/BF01647093](https://doi.org/10.1007/BF01647093). URL: <https://doi.org/10.1007/BF01647093>.
- [DS84] Peter G. Doyle and J. Laurie Snell. *Random Walks and Electric Networks*. 1st ed. Carus Mathematical Monographs 22. original 1984 monograph; digital edition 2012; arXiv v1 2000; arXiv DOI 10.48550/arXiv.math/0001057. Mathematical Association of America, 1984. 159 pp. ISBN: 9780883850244. DOI: [10.5948/UP09781614440222](https://doi.org/10.5948/UP09781614440222). arXiv: [math/0001057](https://arxiv.org/abs/math/0001057). URL: <https://doi.org/10.5948/UP09781614440222>.
- [Höl+26] Johannes Hölzl, Patrick Massot, Casper Putz, and Anne Baanen. *Mathlib module: LinearAlgebra.Matrix.ToLin*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [Hug26] Chris Hughes. *Mathlib module: FiniteDimensional.Basic*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [Lan98] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd ed. Graduate Texts in Mathematics 5. second edition. Springer New York, 1998. 317 pp. DOI: [10.1007/978-1-4757-4721-8](https://doi.org/10.1007/978-1-4757-4721-8). URL: <https://doi.org/10.1007/978-1-4757-4721-8>.
- [Lea26a] Leanprover Community. *Mathlib module: Algebra.BigOperators.Fin*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.

- [Lea26b] Leanprover Community. *Mathlib module: Algebra.Order.BigOperators.Group.Finset*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [Lea26c] Leanprover Community. *Mathlib module: Data.Finset.Max*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [Lea26d] Leanprover Community. *Mathlib module: Data.Fintype.BigOperators*. Version mathlib v4.31.0. mathlib v4.31.0; commit fabf563a7c95. Leanprover Community, 2026.
- [M L97] M. Lothaire. *Combinatorics on Words*. 2nd ed. Cambridge Mathematical Library. second edition; print 1997, online 2009. Cambridge University Press, 1997. 260 pp. ISBN: 9780521599245. DOI: [10.1017/CB09780511566097](https://doi.org/10.1017/CB09780511566097). URL: <https://doi.org/10.1017/CB09780511566097>.
- [Neu23] John von Neumann. “Zur Einführung der transfiniten Zahlen”. In: *Acta Litterarum ac Scientiarum Regiae Universitatis Hungaricae Franciscose Josephinae, Sectio Scientiarum Mathematicarum* 1 (1923). Original German article; official University of Szeged repository scan; no DOI assigned: NOT_ASSIGNED_HISTORICAL_ARTICLE, pp. 199–208. URL: <https://acta.bibl.u-szeged.hu/13294/> (visited on 07/31/2026).
- [Nor97] J. R. Norris. *Markov Chains*. 1st ed. Cambridge Series in Statistical and Probabilistic Mathematics 2. first edition; online 2012. Cambridge University Press, 1997. 237 pp. ISBN: 9780511810633. DOI: [10.1017/CB09780511810633](https://doi.org/10.1017/CB09780511810633). URL: <https://doi.org/10.1017/CB09780511810633>.
- [Pec94] Philip Pechukas. “Reduced Dynamics Need Not Be Completely Positive”. In: *Physical Review Letters* 73.8 (1994). published article, pp. 1060–1062. DOI: [10.1103/PhysRevLett.73.1060](https://doi.org/10.1103/PhysRevLett.73.1060). URL: <https://doi.org/10.1103/PhysRevLett.73.1060>.
- [Pol+18] Felix A. Pollock, César Rodríguez-Rosario, Thomas Frauenheim, Mauro Paternostro, and Kavan Modi. “Non-Markovian quantum processes: Complete framework and efficient characterization”. In: *Physical Review A* 97.1 (2018). published article, p. 012127. DOI: [10.1103/PhysRevA.97.012127](https://doi.org/10.1103/PhysRevA.97.012127). URL: <https://doi.org/10.1103/PhysRevA.97.012127>.
- [SS23] Tomohiro Sugiyama and Kazuhiro Sato. “Kron Reduction and Effective Resistance of Directed Graphs”. In: *SIAM Journal on Matrix Analysis and Applications* 44.1 (2023). published article; arXiv v2 revised 2022-10-28; arXiv v2; arXiv DOI 10.48550/arXiv.2202.12560, pp. 270–292. DOI: [10.1137/22M1480823](https://doi.org/10.1137/22M1480823). arXiv: [2202.12560](https://doi.org/10.1137/22M1480823). URL: <https://doi.org/10.1137/22M1480823>.
- [The26a] The Lean 4 Development Team. *Lean 4 source module: Init.Core*. Version Lean 4.31.0. Lean 4.31.0; commit v4.31.0. Lean FRO, LLC, 2026.
- [The26b] The Lean 4 Development Team. *Lean core module: Init.Data.Fin.Fold*. Version Lean 4.31.0. Lean 4.31.0; commit v4.31.0. Lean FRO, LLC, 2026.
- [The26c] The Lean 4 Development Team. *Lean core module: Init.Data.Rat.Lemmas*. Version Lean 4.31.0. Lean 4.31.0; commit v4.31.0. Lean FRO, LLC, 2026.
- [Zha05] Fuzhen Zhang, ed. *The Schur Complement and Its Applications*. 1st ed. Numerical Methods and Algorithms 4. first edition. Springer New York, 2005. 258 pp. ISBN: 978-0-387-24271-2. DOI: [10.1007/b105056](https://doi.org/10.1007/b105056). URL: <https://doi.org/10.1007/b105056>.